

Web Dynamique côté Serveur

Lecture 3 : Jakarta Contexts and Dependency Injection (CDI)

Simon BERNARD

simon.bernard@univ-rouen.fr



Injection de dépendances

Injection de dépendances

- Dans une application, les composants dépendent souvent les uns des autres
- Considérons la classe **CoffeeService** qui contrôle une machine à café

```
public class CoffeeService {  
    private FilterCoffeeMachine coffeeMachine;  
    public CoffeeService() {  
        this.coffeeMachine = new FilterCoffeeMachine();  
    }  
    public Coffee makeCoffee(int strength, int sugar) {  
        Coffee coffee = coffeeMachine.brewCoffee(strength);  
        coffee.setSugar(sugar);  
        return coffee;  
    }  
}
```

- **CoffeeService** dépend de **FilterCoffeeMachine** : il doit en connaître les constructeurs et les méthodes

Injection de dépendances

- En POO, les dépendances de ce type induisent un couplage fort entre les composants
- Si on veut changer la machine à café (e.g. `EspressoCoffeeMachine`), on doit modifier `CoffeeService`
- Si plein de composants dépendants, modifications en cascade
- Problèmes d'évolutivité et de testabilité
- **Il faut découpler au maximum les composants (classes) pour améliorer la flexibilité et la testabilité**
- Une première solution consiste à utiliser des interfaces et des classes abstraites...

Injection de dépendances

- Pour contrôler plusieurs types de machines, on définit une interface **CoffeeMachine**

```
public interface CoffeeMachine {  
    public Coffee brewCoffee(int strength);  
}
```

- On crée une classe **FilterCoffeeMachine** qui implémente cette interface

```
public class FilterCoffeeMachine implements CoffeeMachine {  
    @Override  
    public Coffee brewCoffee(int strength) {  
        System.out.println("Coffee brewed");  
        return new Coffee("Filter Coffee", strength);  
    }  
    public void addWater() {  
        System.out.println("Water added");  
    }  
    public void addCoffee() {  
        System.out.println("Coffee added");  
    }  
}
```

- On peut imaginer d'autres classes qui implémentent **CoffeeMachine** (e.g. **EspressoCoffeeMachine**)

Injection de dépendances

- On modifie **CoffeeService** pour qu'il utilise l'interface **CoffeeMachine**

```
public class CoffeeService {  
    private CoffeeMachine coffeeMachine;  
    public CoffeeService() {  
        this.coffeeMachine = new FilterCoffeeMachine();  
    }  
    public Coffee makeCoffee(int strength, int sugar) {  
        Coffee coffee = coffeeMachine.brewCoffee(strength);  
        coffee.setSugar(sugar);  
        return coffee;  
    }  
}
```

- Changer l'implémentation est plus simple : plus besoin de connaître les méthodes de **FilterCoffeeMachine**
- Mais la dépendance est toujours présente : il faut instancier **FilterCoffeeMachine**

Injection de dépendances

- Solution : injection de dépendances
- Pour disposer d'une instance de classe qui implémente **CoffeeMachine**, on doit **injecter** cette dépendance via le constructeur de **CoffeeService** (on peut aussi le faire via un setter).

```
public class CoffeeService {  
    private CoffeeMachine coffeeMachine;  
    public CoffeeService(CoffeeMachine coffeeMachine) {  
        this.coffeeMachine = coffeeMachine;  
    }  
    public Coffee makeCoffee(int strength, int sugar) {  
        Coffee coffee = coffeeMachine.brewCoffee(strength);  
        coffee.setSugar(sugar);  
        return coffee;  
    }  
}
```

- Responsabilité de la création de l'instance déléguée à l'extérieur de **CoffeeService**
- On peut changer l'implémentation sans impacter **CoffeeService**

Injection de dépendances

- Ce principe d'injection de dépendances rend le code plus flexible et testable
- Nécessite de fournir les dépendances requises à la création de **CoffeeService**
- La multiplication des composants rend la gestion des dépendances complexe

```
CoffeeType type = new CoffeeType("Medium Fine", 0.5);  
CoffeeConfiguration config = new CoffeeConfiguration(type, 0.7, 550);  
FilterCoffeeMachine coffeeMachine = new FilterCoffeeMachine(config);  
CoffeeService coffeeService = new CoffeeService(coffeeMachine);  
Coffee coffee = coffeeService.makeCoffee(2, 1);
```

- Solution : utiliser un framework d'injection de dépendances, e.g. Jakarta CDI
- C'est le conteneur CDI qui crée et gère les instances des classes
- Le développeur se contente de déclarer les dépendances requises

Jakarta Contexts and Dependency Injection

- Annotation `@Inject` pour déclarer les dépendances

```
public class CoffeeService {  
    private CoffeeMachine coffeeMachine;  
    @Inject  
    public CoffeeService(CoffeeMachine coffeeMachine) {  
        this.coffeeMachine = coffeeMachine;  
    }  
    ...  
}
```

ou injection par champs :

```
public class CoffeeService {  
    @Inject  
    private CoffeeMachine coffeeMachine;  
    ...  
}
```

Jakarta Contexts and Dependency Injection

- Le conteneur CDI recherche les classes qui correspondent aux dépendances déclarées
- Il crée des instances de ces classes et les injecte dans les composants qui en ont besoin
- Pour gérer le cycle de vie des instances, le conteneur doit connaître la portée de l'instance
- Si plusieurs classes candidates, le conteneur doit pouvoir choisir celle qui correspond au contexte d'injection
- **Il faut configurer les classes injectables : les CDI managed beans**
- Note : CDI permet aussi d'injecter des ressources (e.g. des connexions à une base de données)

```
@Resource(name = "jdbc/myDataSource")
private DataSource dataSource;
public Connection getConnection() throws SQLException {
    return dataSource.getConnection();
}
```

Nous ne couvrirons pas ce cas d'usage dans ce cours.

CDI Managed Beans

CDI managed beans

- Un *managed bean* est une classe Java qui peut être gérée par le conteneur CDI
- Elle doit :
 - Être **public et instantiable** (non abstraite)
 - Avoir un **constructeur public sans argument ou avec des paramètres annotés** avec **@Inject**
 - Être **dans un package scanné par le conteneur CDI**
- Elle est souvent annotée pour configurer son comportement (mais pas obligatoire)
- Toutes les classes qui respectent ces contraintes sont des *managed bean* potentiels, c'est-à-dire qu'elles peuvent être découvertes et gérées automatiquement par le conteneur CDI
- Note : englobe les JavaBeans (entre autres)

CDI managed beans

- Le conteneur CDI associe à chaque *managed bean* plusieurs attributs :
 - un ensemble de types de bean
 - un ensemble de *qualifiers*
 - une portée
 - optionnellement, un nom de bean
 - un ensemble d'intercepteurs

CDI managed beans

Types de beans

- Un bean a un ou plusieurs types à l'injection desquels il est candidat
- Exemple : `FilterCoffeeMachine` a les types `CoffeeMachine`, `FilterCoffeeMachine` et `java.lang.Object`
- Ces types peuvent être explicitement restreint

```
@Typed(CoffeeMachine.class)
public class FilterCoffeeMachine implements CoffeeMachine {
    ...
}
```

- Ici, les types de bean sont `CoffeeMachine` et `Object`
- Signifie que `FilterCoffeeMachine` est injectable en tant que `CoffeeMachine` (ou `Object`) uniquement

CDI managed beans

Qualifiers

- Si plusieurs types de bean correspondent à une dépendance : Exception au déploiement

```
public class FilterCoffeeMachine implements CoffeeMachine { ... }  
public class EspressoCoffeeMachine implements CoffeeMachine { ... }
```

```
public class CoffeeService {  
    private CoffeeMachine coffeeMachine;  
    @Inject  
    public CoffeeService(CoffeeMachine coffeeMachine) { // Erreur: plusieurs beans correspondants  
        this.coffeeMachine = coffeeMachine;  
    }  
}
```

CDI managed beans

Qualifiers

- Pour désambiguer, on utilise des *qualifiers*

```
@Filter
public class FilterCoffeeMachine implements CoffeeMachine { ... }
@Espresso
public class EspressoCoffeeMachine implements CoffeeMachine { ... }
```

- Permet de préciser le type de machine à café souhaité lors de l'injection

```
public class CoffeeService {
    private CoffeeMachine coffeeMachine;
    @Inject
    public CoffeeService(@Filter CoffeeMachine coffeeMachine) { // OK: un seul bean correspondant
        this.coffeeMachine = coffeeMachine;
    }
}
```


CDI managed beans

Qualifiers

- Les *qualifiers* de l'exemple précédent sont à définir

```
@Qualifier
@Retention(RUNTIME)
@Target({TYPE, METHOD, FIELD, PARAMETER})
public @interface Filter { }
```

```
@Qualifier
@Retention(RUNTIME)
@Target({TYPE, METHOD, FIELD, PARAMETER})
public @interface Espresso { }
```

- **@Qualifier**, **@Retention** et **@Target** sont des meta-annotations, c'est-à-dire des annotations d'annotations
- **@Retention(RUNTIME)** indique que le *qualifier* est disponible à l'exécution (pas le cas par défaut)
- **@Target({TYPE, METHOD, FIELD, PARAMETER})** précise où le *qualifier* peut être utilisé

CDI managed beans

Portée des beans

- La portée détermine la durée de vie et la visibilité des beans
- Annotation de classe : `@ApplicationScoped`, `@SessionScoped`, `@RequestScoped`
- Par défaut, la portée est `@Dependent` = une instance par injection
- Exemple : servlet qui utilise `CoffeeService`

```
@WebServlet("/coffee")
public class CoffeeServlet extends HttpServlet {
    @Inject
    private CoffeeService coffeeService;
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        Coffee coffee = coffeeService.makeCoffee(3, 2);
        request.setAttribute("coffee", coffee);
        RequestDispatcher dispatcher = request.getRequestDispatcher("/coffee.jsp");
        dispatcher.forward(request, response);
    }
}
```

- Question : une nouvelle instance de `CoffeeService` à chaque requête ? pour chaque client ? au déploiement ?

CDI managed beans

Portée des beans

- **CoffeeService** doit spécifier sa portée (requête, session ou application)
- Le conteneur cherche une instance dans cette portée
- Si elle existe, elle est réutilisée. Si elle n'existe pas, une nouvelle instance est créée
- Exemple :

```
@SessionScoped
public class CoffeeService implements Serializable{
    ...
}
```

Une instance de **CoffeeService** est créée à la réception par **CoffeeServlet** de la première requête dans une session utilisateur, puis réutilisée pour toutes les requêtes suivantes dans la même session

- Les instances sont détruites quand la portée prend fin (e.g. quand la session expire)
- Note : **CoffeeService** doit implémenter **Serializable** pour être stocké dans la session

CDI managed beans

Nom de bean

- Un bean peut avoir un nom, le rendant accessible dans un environnement non typé (e.g. EL)
- Annotation du bean avec `@Named`
- Le nom par défaut est le nom de la classe avec une minuscule

```
@Named // Nom par défaut: "coffeeService"
@RequestScoped
public class CoffeeService {
    @Inject
    private CoffeeMachine coffeeMachine;
    public String getName() {
        return "The M2 SIME coffee shop";
    }
    ...
}
```

- On peut spécifier un nom différent avec l'annotation `@Named("myCoffeeShop")`

```
<h1>Welcome to ${ myCoffeeShop.name }</h1>
```

- Pas d'intérêt quand on utilise des servlets (e.g. architecture MVC du chapitre précédent)

Intercepteurs

- Les intercepteurs permettent de déclencher des actions avant ou après l'instanciation d'un bean ou l'exécution d'une de ses méthodes
- Exemple : utilisation de **CoffeeService** requiert une authentification, c'est-à-dire qu'à chaque appel de la méthode **makeCoffee**, on vérifie que l'utilisateur est authentifié et/ou lui proposer de s'authentifier avant
- Pour ça, on crée un intercepteur

```
@Interceptor
public class AuthenticationInterceptor {
    @AroundInvoke
    public Object checkAuthentication(InvocationContext context) throws Exception {
        if (!isUserAuthenticated()) {
            // Rediriger vers la page de connexion
        }
        return context.proceed();
    }
}
```

- La méthode devra être exécutée automatiquement avant l'exécution de la méthode interceptée

Intercepteurs

- Pour utiliser l'intercepteur : annotation `@Interceptors(AuthenticationInterceptor.class)` sur la méthode ciblée

```
@RequestScoped
public class CoffeeService {
    ...
    @Interceptors(AuthenticationInterceptor.class)
    public Coffee makeCoffee(int strength, int sugar) {
        ...
    }
}
```

- Autre annotation possible : `AroundConstruct`
- Dans ce cas, on annote la classe cible et l'intercepteur est déclenché à l'instanciation

Intercepteurs

Annotations `@PostConstruct` et `@PreDestroy`

- Si un bean est injecté ET contient des injections par champs :
 1. le conteneur crée l'instance avec le constructeur par défaut
 2. le conteneur injecte ses dépendances
- Les injections par champs ne sont pas effectuées quand le constructeur est exécuté
- `@PostConstruct` et `@PreDestroy` pour exécuter du code après l'injection et avant la destruction de l'instance :

```
@RequestScoped
public class CoffeeService {
    @Inject
    private CoffeeMachine coffeeMachine;
    @PostConstruct
    public void init() {
        this.coffeeMachine.preheating();
    }
}
```

- L'injection par constructeur évite ce problème

Méthodes *Producer*

- Les méthodes *producer* permettent de créer des instances de beans de manière programmatique
- Annotation **@Produces** pour une méthode qui crée un bean

```
@ApplicationScoped
public class CoffeeSelector {
    private final static int FILTER = 1;
    private final static int EXPRESSO = 2;
    private int coffeeMachineType = FILTER; // default value
    @Produces @Selected
    public CoffeeMachine getCoffeeMachine() {
        if (this.coffeeMachineType == EXPRESSO) {
            return new ExpressCoffeeMachine();
        }
        return new FilterCoffeeMachine();
    }
}
```

- **@Selected** est un *qualifier*

Méthodes *Producer*

- Injection de `@Selected CoffeeMachine`

```
@RequestScoped
public class CoffeeService {
    @Inject @Selected
    private CoffeeMachine coffeeMachine;
    public void makeCoffee() {
        this.coffeeMachine.brewCoffee();
    }
}
```

- Le conteneur appelle `getCoffeeMachine()` pour obtenir une instance de `CoffeeMachine`
- Pas de conflit avec `FilterCoffeeMachine` grâce à `@Selected`
- Le type du bean injecté est déterminé à l'exécution (et peut changer)
- Note : possible ici de définir la méthode *producer* dans `CoffeeService` directement

Packaging

- Les beans peuvent être empaquetés dans des archives **JAR** ou **WAR**
- Une archive est considérée comme un module CDI si elle contient un fichier **beans.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
                           https://jakarta.ee/xml/ns/jakartaee/beans_3_0.xsd"
       version="3.0" bean-discovery-mode="all">
    ...
</beans>
```

- **beans.xml** doit être placé dans **WEB-INF** si **WAR**, **META-INF** si **JAR**
- **bean-discovery-mode** spécifie le mode de découverte des beans : toutes les classes (**all**), seulement les classes annotées (**annotated**) ou aucune classe (**none**)

Packaging

- Le conteneur CDI scanne automatiquement les classes de toutes les archives (**WAR** ou **JAR**) d'un projet
- Deux cas de figures :
 - **beans.xml** est présent dans l'archive et non vide : applique le **bean-discovery-mode** spécifié
 - **beans.xml** est absent ou vide : seuls les classes annotées sont des **beans** potentiels
- **beans.xml** peut spécifier des exclusions de classes ou de packages :

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/beans_3_0.xsd"
  version="3.0" bean-discovery-mode="all">
<exclude name="com.example.rest.*" />
<exclude name="com.example.faces.**">
  <if-class-not-available name="jakarta.faces.context.FacesContext"/>
</exclude>
</beans>
```

Live coding: Le projet `coffeeapp`